Connecting to Databases Using JDBC

Ehsan Khamespanah

Spring 2024

How to Store Data

- Saving records of data in a file
  - Comma separated data

Reservations.txt

```
Ali,Alavi,Beijing,2020/06/07,
Vali,Valavi,Paris, …
```

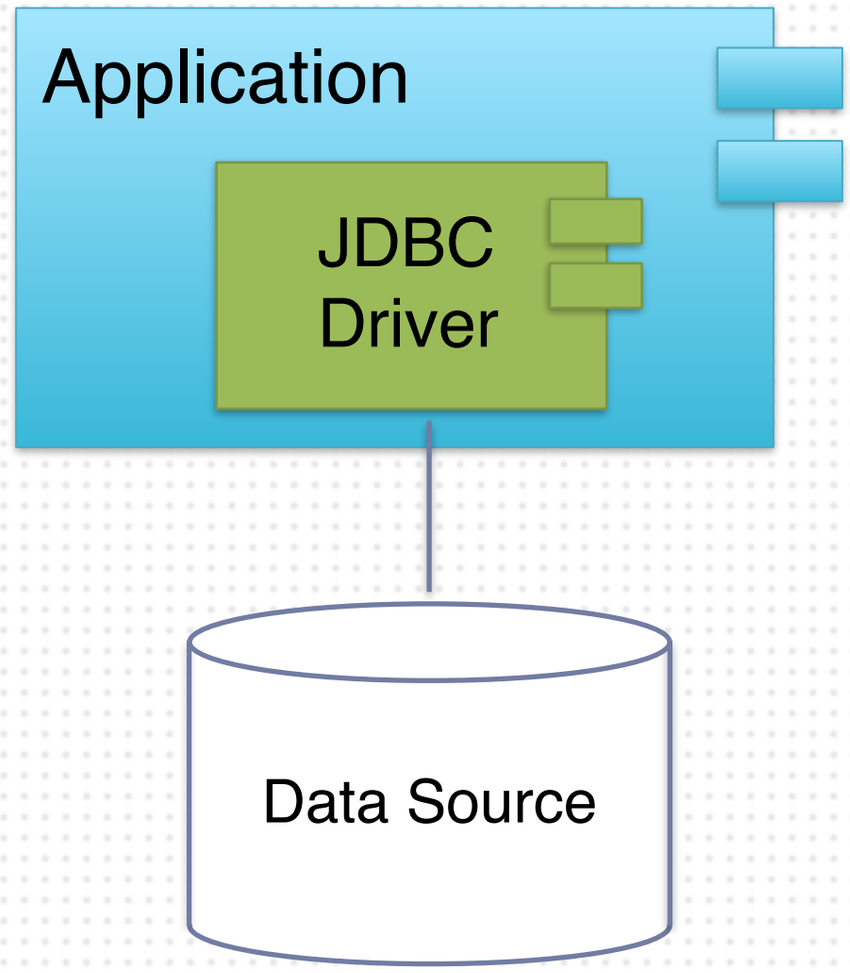
  - Fixed length record area

Reservations.txt

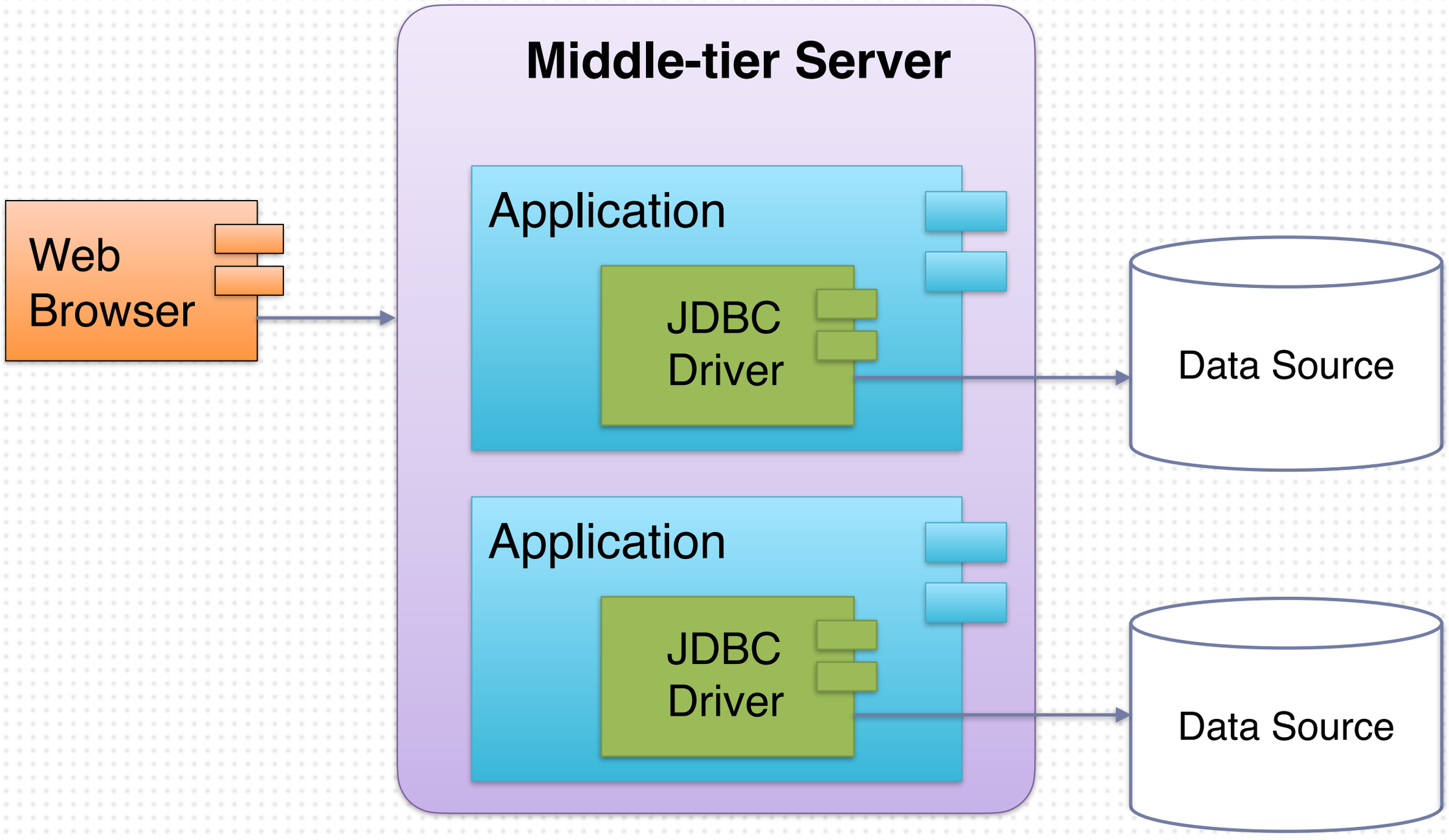
```
Ali      ;Alavi      ;Beijing      ;2020/06/07;
Vali     ;Valavi     ;Paris        ;…
```

- How to update/delete data? or record structure?
- How to search based on complex queries?

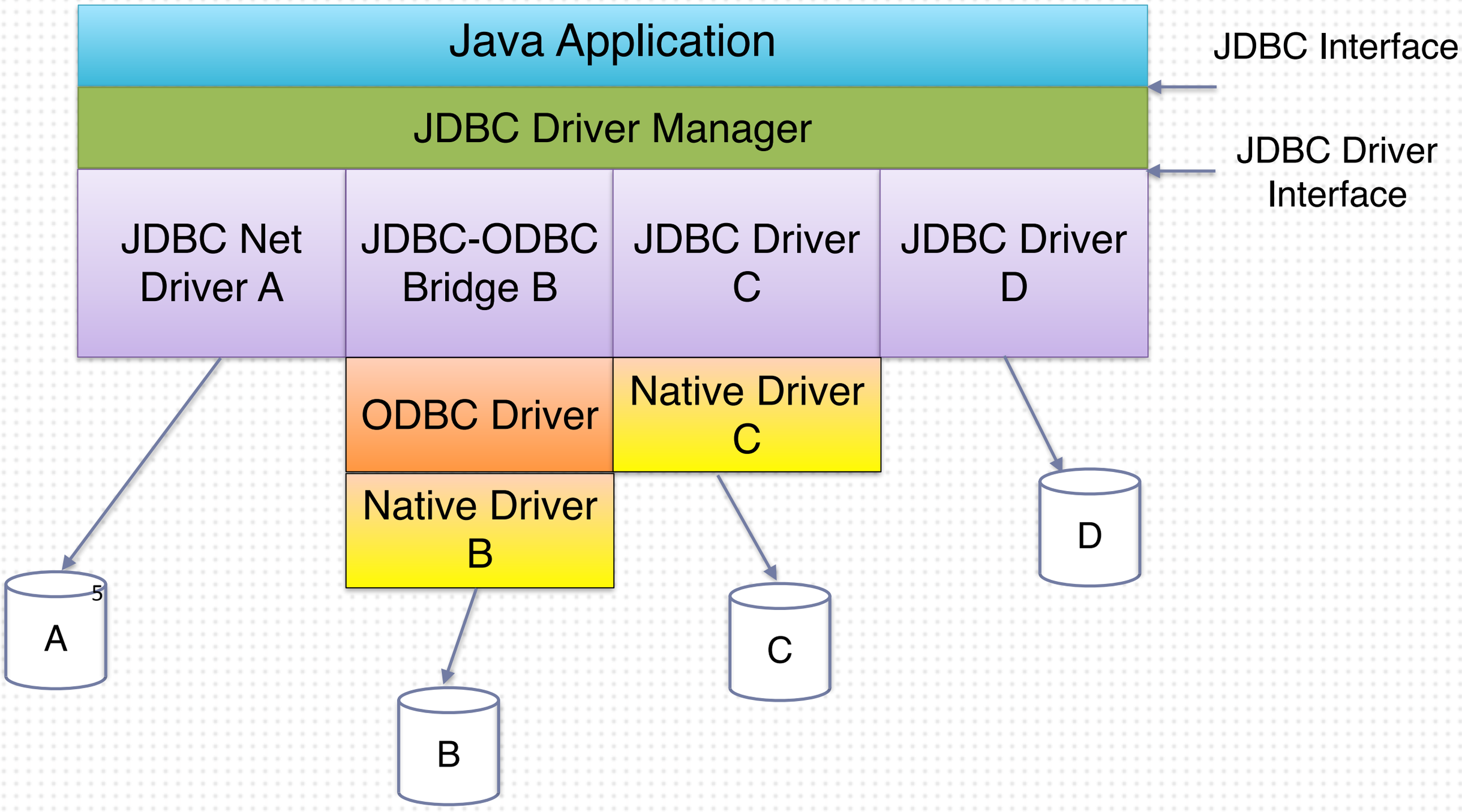
Two-Tier Model



Three-Tier Model



Complete JDBC Architecture



Establishing a Connection

- Using DriverManager
  - Loading driver class manually
  - Providing connection URL

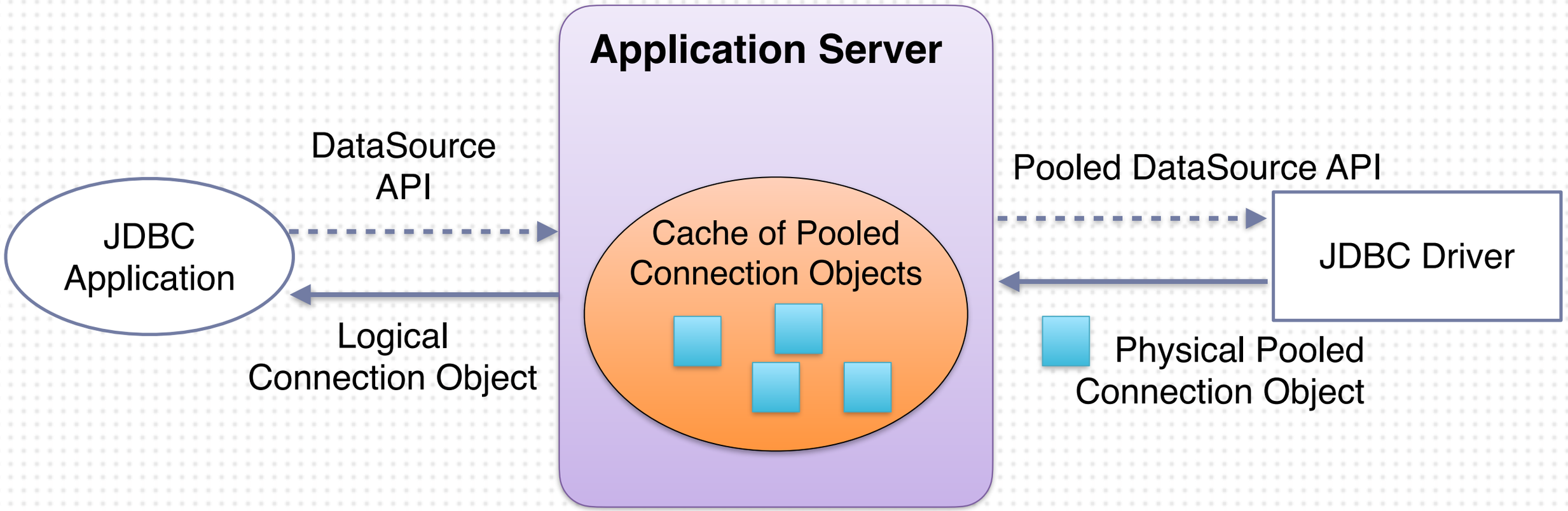
```
Properties props = new Properties();
props.put("user", "ali");
props.put("password", "1234");
Connection con = DriverManager.getConnection(
     “jdbc:mysql://localhost:3306/mydb",props);
```

Establishing a Connection

- Using DriverManager
- Using DataSource
  - Establishing the connection using a symbolic name
  - Details are transparent to the application

```
// Get the initial JNDI naming context
Context ctx = new InitialContext();
// Get DataSource object associated with the logical name
"jdbc/AcmeDB" and use it to obtain a database connection
DataSource ds = (DataSource)ctx.lookup("jdbc/AcmeDB");
Connection con = ds.getConnection("user", "pwd");
```

Connection Pooling



Executing SQL Queries

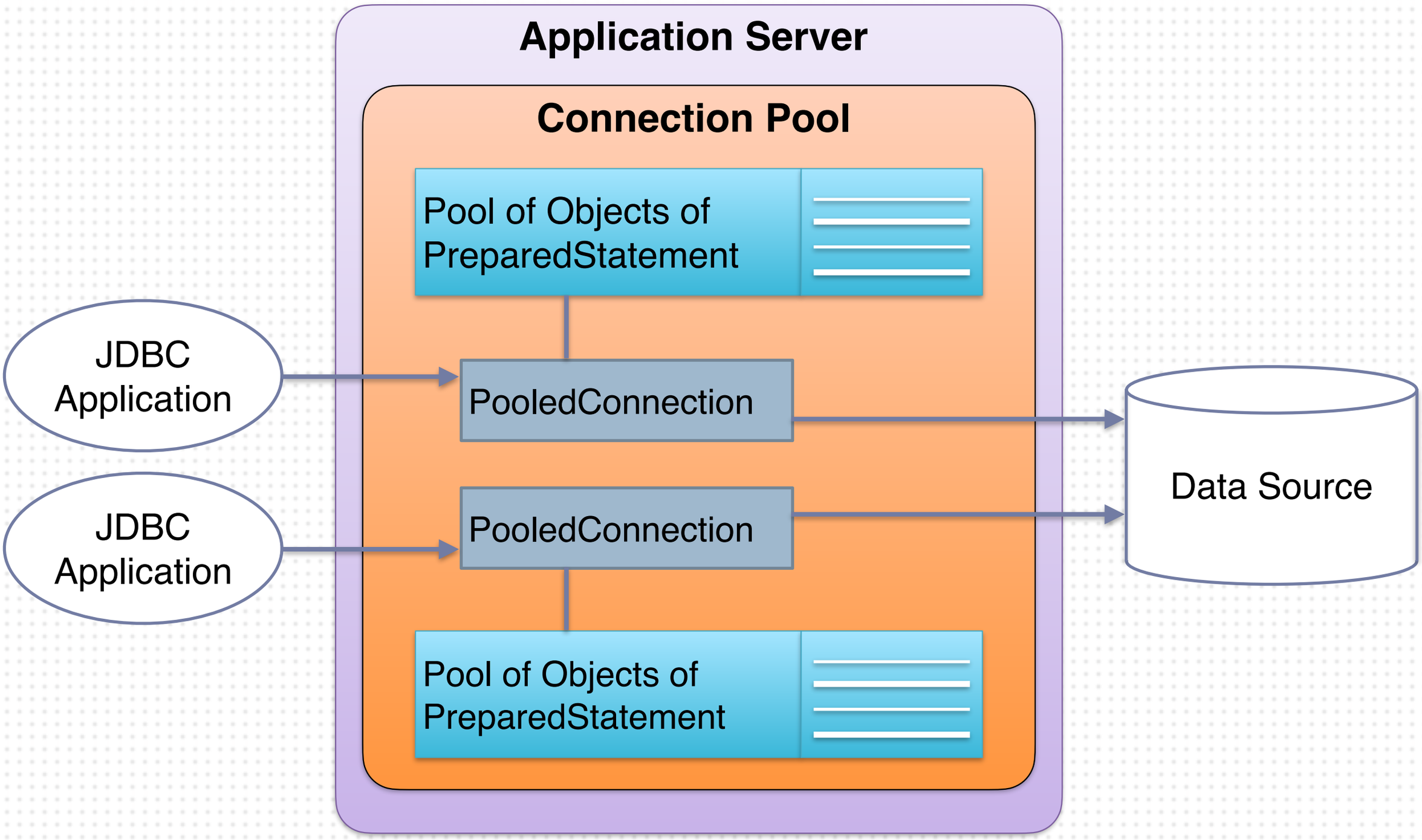
- Statements

```
Statement stmt = con.createStatement();
String sql = "INSERT INTO Registration " +
             "VALUES (100, 'Ali', ‘Alavi’)";
stmt.executeUpdate(sql);
```

- Prepared Statements

```
PreparedStatement pstmt = con.prepareStatement(
        "UPDATE EMPLOYEES SET CAR_NUMBER = ? " +
        "WHERE EMPLOYEE_NUMBER = ?");
pstmt.setInt(1, carNo);
pstmt.setInt(2, empNo);
pstmt.executeUpdate();
```

Pooling Statements



Prepared Statements

```
PreparedStatement pstmt =
   conn.prepareStatement("SELECT AUTHOR, " +
   "TITLE FROM BOOKLIST WHERE SECTION = ?");
for (int i = 1; i <= maxSectionNumber; i++) {
   pstmt.setInt(1, i);
   ResultSet rs = pstmt.executeQuery();
   while (rs.next()) {
      // process the record
   }
   rs.close();
}
pstmt.close();
```

Callable Statements

Callable Statements

- Defining a stored procedure

```
CREATE OR REPLACE PROCEDURE insertEmployee
(in_id IN EMPLOYEE.EMPID%TYPE,
 in_name IN EMPLOYEE.NAME%TYPE,
 in_role IN EMPLOYEE.ROLE%TYPE,
 in_city IN EMPLOYEE.CITY%TYPE,
 in_country IN EMPLOYEE.COUNTRY%TYPE,
 out_result OUT VARCHAR2)
AS
BEGIN
  INSERT INTO EMPLOYEE (EMPID, NAME, ROLE, CITY, COUNTRY)
  values (in_id,in_name,in_role,in_city,in_country);
  commit;

  out_result := 'TRUE';

EXCEPTION
  WHEN OTHERS THEN
  out_result := 'FALSE';
  ROLLBACK;
END;
```

Callable Statements

- Defining a stored procedure
- Calling a stored procedure

```
con = DBConnection.getConnection();
stmt = con.prepareCall("{call insertEmployee(?, ?, ?, ?, ?, ?)}”);
stmt.setInt(1, id);
stmt.setString(2, name);
stmt.setString(3, role);
stmt.setString(4, city);
stmt.setString(5, country);

//register the OUT parameter before calling the stored procedure
stmt.registerOutParameter(6, java.sql.Types.VARCHAR);

stmt.executeUpdate();

//read the OUT parameter now
String result = stmt.getString(6);

System.out.println("Employee Record Save Success::” + result);
```

Batch Updates

- Instead of multiple independent updates, a set of updates maybe sent for applying

```
con.setAutoCommit(false);

Statement stmt = con.createStatement();
stmt.addBatch("INSERT INTO employees VALUES (1000, 'Joe Jones')");
stmt.addBatch("INSERT INTO departments VALUES (260, 'Shoe')");
stmt.addBatch("INSERT INTO emp_dept VALUES (1000, 260)");

int[] updateCounts = stmt.executeBatch();
```

Batch Update - Prepared Statements

- Cheaper way of executing a set of SQL queries

```
PreparedStatement stmt = con.prepareStatement(
   "INSERT INTO employees VALUES (?, ?)");
stmt.setInt(1, 2000);
stmt.setString(2, "Kelly Kaufmann");
stmt.addBatch();
stmt.setInt(1, 3000);
stmt.setString(2, "Bill Barnes");
stmt.addBatch();

int[] updateCounts = stmt.executeBatch();
```

Result Sets and Cursors

select * from student

a	Name	Mid	Final
1004	Xi	10.3	12
--	--	--	--
--	--	--	—

- Moving cursors
  - Forward & Backward
  - Relative & Absolute

Result Sets and Cursors

- Navigating the results using “next” method

```
String sql = "SELECT id, first, last, age FROM
Registration";
ResultSet rs = stmt.executeQuery(sql);

//Extract data from result set
while(rs.next()){
  //Retrieve by column name
  int id  = rs.getInt("id");
  int age = rs.getInt("age");
  String first = rs.getString("first");
  String last = rs.getString("last");
  //……
}
rs.close();
```

Cursor Movement

- next()
- previous()
- first()
- last()
- beforeFirst()
- afterLast()
- relative(int rows)
- absolute(int row)

ResultSets Types

- TYPE_FORWARD_ONLY
  - Not scrollable
- TYPE_SCROLL_SENSITIVE
  - Scrollable,
  - Reflecting updates during scroll
- TYPE_SCROLL_INSENSITIVE
  - Scrollable,
  - Not reflecting updates during scroll

ResultSet Concurrency

- CONCUR_READ_ONLY
- CONCUR_UPDATABLE

- Determining concurrency:
  - ResultSet.getConcurrency()

ResultSet Holdability

- If cursors are closed after commit

- HOLD_CURSORS_OVER_COMMIT
- HOLD_CURSORS_AT_COMMIT

- Determining holdability:
  - ResultSet.getHoldability()

Specifying Parameters

- Parameters can be defined for each statement

```
Statement stmt = con.createStatement(
   ResultSet.TYPE_SCROLL_INSENSITIVE,
   ResultSet.CONCUR_READ_ONLY,
   ResultSet.CLOSE_CURSORS_AT_COMMIT);
```

Updating a Row

- Update data during navigation

```
Statement stmt = con.createStatement(
   ResultSet.TYPE_FORWARD_ONLY,
   ResultSet.CONCUR_UPDATABLE);

ResultSet rs = stmt.executeQuery(
   "select author from booklist " +
   "where isbn = 140185852");
rs.next();
rs.updateString("author", "Zamyatin, E. I.");
rs.updateRow();
```

Transactions in Databases

- Sometimes, we may want to commit the transaction after the execution of some more SQL statements.
- If we get an exception in the transaction, we can rollback() changes and make it like before
- Transactions provide ACID properties:
  - A–Atomicity -> all or none
  - C–Consistency -> consistent state after any transaction
  - I– Isolation -> transaction are isolated from each other
  - D–Durability -> If the transaction is committed once, it will remain always committed

Transactions in Databases

- Sometimes, we may want to commit the transaction after the execution of some more SQL statements
- If v
  rol
- Tra
  -
  -
  -
  -

```
try {
   //Assume a valid connection object conn
   conn.setAutoCommit(false);
   Statement stmt = conn.createStatement();

   String SQL = "INSERT INTO Employees  " +
                "VALUES (106, 20, 'Rita', 'Tez')";
   stmt.executeUpdate(SQL);
   //Submit a malformed SQL statement that breaks
   SQL = "INSERTED IN Employees  " +
                "VALUES (107, 22, 'Sita', 'Singh')";
   stmt.executeUpdate(SQL);
   // If there is no error.
   conn.commit();
} catch(SQLException se){
   // If there is any error.
   conn.rollback();
}
```

tion

her

e, it

Transactions in Databases

- Controlling transactions using three functions

```
con.setAutoCommit(boolean);
con.commit();
con.rollback();
```

- We will see later about
  - Transaction Isolation Levels
  - Distributed Transactions

Type Conversions

JDBC Types	Java Type
CHAR	String
VARCHAR	String
LONGVARCHAR	String
NUMERIC	java.math.BigDecimal
DECIMAL	java.math.BigDecimal
BIT	boolean
Boolean	boolean
TINYINT	byte
SMALLINT	short

Blobs

- Complex data (e.g. image)

```
String query = "INSERT INTO SampleTable(Name,image) VALUES (?, ?)";
PreparedStatement pstmt = con.prepareStatement(query);
pstmt.setString(1, "sample image");
FileInputStream fin = new FileInputStream(“~/Data/cat.jpg”);
pstmt.setBlob(2, fin);
pstmt.execute();
```

```
ResultSet rs = stmt.executeQuery("select * from SampleTable");
int i = 1;
while(rs.next()) {
    System.out.println(rs.getString("Name"));
    Blob blob = rs.getBlob("Image");
    byte byteArray[] = blob.getBytes(1,(int)blob.length());
    FileOutputStream outPutStream = new
    FileOutputStream(“~/Data/blob_output”+i+".jpg");
    outPutStream.write(byteArray);
   i++;
}
```